



Part I: Wyckoff ChatBot Module



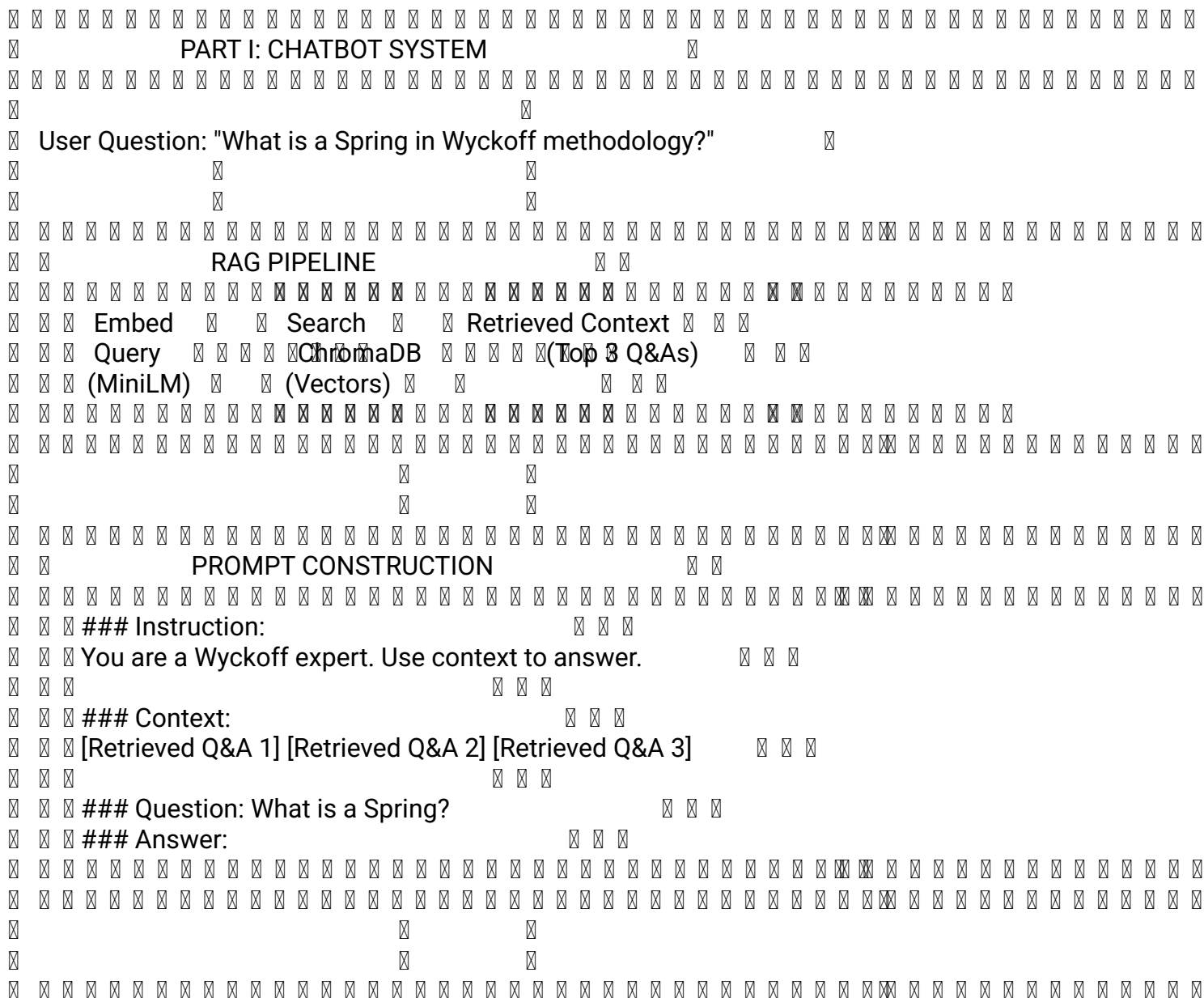
What is Part I (ChatBot)?

Part I of the Wyckoff Trader project is the **AI ChatBot Module** that:

1. Fine-tunes LLaMA-2-7B on Wyckoff methodology Q&A pairs
2. Builds a RAG pipeline for accurate context retrieval
3. Combines both to generate expert-level answers
4. Provides a chat interface for users to ask questions



High-Level Architecture



FINE-TUNED LLaMA-2-7B
 Base LoRA Generated
 LLaMA + Adapters = Response
 (frozen) (trained)
 "A Spring is a false breakdown below support that traps sellers, providing a low-risk entry opportunity. It occurs in Phase C..."

 The Two Main Components

Component 1: Fine-Tuned LLM (LLaMA-2-7B + LoRA)

FINE-TUNING EXPLAINED
 WHAT IS FINE-TUNING?
 Teaching a pre-trained model new knowledge or skills by training it on domain-specific data (our Wyckoff Q&A pairs).
 BEFORE Fine-tuning:
 LLaMA-2-7B: General knowledge model
 • Knows general facts, language, reasoning
 • Does NOT know Wyckoff methodology specifically
 • May give generic or incorrect trading answers
 AFTER Fine-tuning:
 LLaMA-2-7B + LoRA (Wyckoff Expert):
 • Retains general knowledge
 • NOW knows Wyckoff methodology deeply
 • Speaks in Wyckoff terminology
 • Gives accurate, specific trading answers

Component 2: RAG (Retrieval Augmented Generation)

RAG EXPLAINED

WHAT IS RAG?

A technique that RETRIEVES relevant information from a database and AUGMENTS the prompt before GENERATING a response.

WHY USE RAG?

- Provides specific facts the model might not remember
- Reduces hallucination (making up information)
- Keeps responses grounded in your actual data
- Can update knowledge without retraining

RAG WORKFLOW:

User Query

1. EMBED Convert query to 384-dimensional vector (MiniLM) "What is Spring?" [0.12, -0.34, 0.56, ...]

2. SEARCH Find most similar vectors in database (ChromaDB) Using cosine similarity

3. RETRIEVE Get top 3 most relevant Q&A pairs (Top-K) With similarity scores

4. AUGMENT Add retrieved context to prompt (Prompt) Before sending to LLM

5. GENERATE LLM generates answer using context (LLaMA) More accurate and grounded



Why Use BOTH Fine-tuning AND RAG?

FINE-TUNING vs RAG vs BOTH

FINE-TUNING ONLY

- Model learns Wyckoff "style" and terminology

- Responds naturally in domain language

- May hallucinate specific facts

- Cannot cite sources

- Hard to update knowledge (need to retrain)

Example Response:

"A Spring occurs when price breaks support..." (might be wrong)

RAG ONLY

- Always grounded in actual data

- Can cite sources

- Easy to update (just add documents)

- Generic language style

- May not synthesize information well

- Depends heavily on retrieval quality

Example Response:

"According to the document, a Spring is..." (robotic)

FINE-TUNING + RAG (OUR APPROACH)

- Model speaks naturally in Wyckoff terminology

- Responses grounded in actual data

- Can synthesize and explain concepts

- Reduced hallucination

- Best of both worlds!

Example Response:

"A Spring is a critical Phase C event where price temporarily breaks below support to shake out weak holders. This false breakdown traps sellers and provides smart money with a low-risk entry point. Look for low volume on the breakdown to confirm it's a genuine Spring rather than real selling..."



What is LoRA (Low-Rank Adaptation)?

LoRA EXPLAINED

THE PROBLEM:

LLaMA-2-7B has 7 BILLION parameters

Full fine-tuning would require:

- 28+ GB GPU memory
- Hours/days of training
- Huge storage for model

THE SOLUTION: LoRA

Only train small "adapter" matrices, keep base model frozen

Original Weight Matrix W (frozen, not trained)

7B parameters

☐ NOT CHANGED

(frozen)

LoRA Adapter (small, trained)

$A \times B$ Only ~50MB trained!

 $(r \times d)$ $(d \times r)$

☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒ ☒ ☐ ☐ ☐ ☐ ☐

$r = 16$ (rank), $d =$ hidden dimension

$$\text{Final Output} = W \times \text{input} + (A \times B) \times \text{input}$$

BENEFITS:

- Train only 0.1% of parameters (~7M vs 7B)
- Fits in 8GB GPU memory (with 4-bit quantization)
- Training takes 15-30 minutes
- Adapter files only ~50MB
- Can swap adapters for different tasks

4-Bit Quantization

4-BIT QUANTIZATION

[illegible]

Reducing the precision of model weights to use less memory

- Normal (FP16): 16 bits per weight ~14GB for 7B model

- 4-bit (NF4): 4 bits per weight ~3.5GB for 7B model

[illegible]

 NF4 Weight: 0.12 (4 bits, lower precision) 

Memory Savings: 75% reduction!

☒ ☒	Quality Loss: Minimal (especially with NF4 format)	☒ ☒
-----	--	-----

[illegible]☐ • load_in_4bit = True ☐

- `bnb_4bit_quant_type = "nf4"` (normalized float 4-bit)

- `bnb_4bit_compute_dtype = float16` (compute in higher precision)

- bnb_4bit_use_double_quant = True (extra compression)

MODEL SETUP:

Base Model: meta-llama/Llama-2-7b-hf

Quantization: 4-bit NF4

LoRA Rank (r): 16

LoRA Alpha: 32

LoRA Dropout: 0.1

Target Modules: q_proj, k_proj, v_proj, o_proj,

gate_proj, up_proj, down_proj

 TRAINING SETUP:

Epochs: 3

Batch Size: 2

🔍 🔍 Gradient Accumulation: 8 🔍 🔍

Effective Batch Size: 16 (2 × 8)

Learning Rate: 1e-4

☒ ☒ Max Sequence Length: 512 tokens ☒ ☒

Optimizer: paged_adamw_8bit

LR Scheduler: cosine
 Warmup Ratio: 0.1
 WHAT EACH PARAMETER DOES:
 • LoRA Rank (r=16): Size of adapter matrices. Higher = more capacity
 • LoRA Alpha (32): Scaling factor. Usually 2× rank
 • Dropout (0.1): Regularization to prevent overfitting
 • Target Modules: Which layers to adapt (attention + MLP)
 • Gradient Accumulation: Simulate larger batch with less memory
 • Cosine LR: Learning rate decreases smoothly

Training Data Format

TRAINING DATA FORMAT
 INPUT CSV FORMAT:

Questions	Answers	Label
What is a Spring?	A Spring is a false...	Timing
What are three laws?	Wyckoff's three laws...	Strategy
When to enter SC?	After SC, wait for...	Timing

 CONVERTED TO INSTRUCTION FORMAT:

```

    ### Instruction:
    You are an expert on Richard Wyckoff's trading methodology.
    Answer the following question accurately and thoroughly.

    ### Question:
    What is a Spring in Wyckoff methodology?

    ### Answer:
    A Spring is a false breakdown below support that occurs in
    Phase C of accumulation. It is designed to shake out weak
    holders and trap short sellers before the markup phase begins.
    A successful Spring shows low volume on the breakdown,
    indicating lack of real selling pressure...
  
```

 WHY THIS FORMAT?
 • "Instruction" tells model its role
 • "Question" clearly marks user input
 • "Answer" teaches model expected output format
 • Model learns to generate after "### Answer:"

🔍 RAG Deep Dive

Embedding Model (MiniLM)

📌 **EMBEDDING MODEL**

📌 **MODEL:** sentence-transformers/paraphrase-MiniLM-L6-v2

📌 **WHAT IT DOES:**

📌 Converts text into a numerical vector (array of numbers) that captures the MEANING of the text.

📌 **Input Text:** "What is a Spring?"

📌 **Output Vector:** [0.12, -0.34, 0.56, -0.78, ..., 0.23]

📌 **384 dimensions**

📌 **KEY PROPERTY - SEMANTIC SIMILARITY:**

📌 Similar meanings → Similar vectors → Small distance

📌 "What is a Spring?" → Distance: 0.15 (SIMILAR!)

📌 "Explain Spring event" → Distance: 0.85 (DIFFERENT)

📌 "What is a Spring?" → Distance: 0.85 (DIFFERENT)

📌 "What is the weather?" → Distance: 0.85 (DIFFERENT)

📌 **WHY MiniLM?**

- Small and fast (22M parameters)

- Good quality embeddings

- Works well for sentence similarity

- Only 384 dimensions (efficient)

Vector Database (ChromaDB)

VECTOR DATABASE

WHAT IS A VECTOR DATABASE?

A database optimized for storing and searching vectors (embeddings) using similarity measures like cosine similarity.

OUR ChromaDB STRUCTURE:

Collection: "wyckoff_qa"

ID	Embedding (384-dim)	Document	Metadata
----	---------------------	----------	----------

qa_0	[0.12, -0.34, ...]	"What is answer: Spring?"	"A..."
------	--------------------	---------------------------	--------

qa_1	[0.45, 0.23, ...]	"Three answer: laws?"	"The..."
------	-------------------	-----------------------	----------

qa_2	[-0.11, 0.67, ...]	"When to answer: enter?"	"After..."
------	--------------------	--------------------------	------------

...	...	...	...
-----	-----	-----	-----

qa_199	[0.33, -0.21, ...]	"Volume answer: patterns?"	"In..."
--------	--------------------	----------------------------	---------

Total: 200+ Q&A pairs indexed

Cosine Similarity

COSINE SIMILARITY

FORMULA:

$$\text{similarity} = \frac{A \cdot B}{\|A\| \times \|B\|} = \frac{\sum(A_i \times B_i)}{\sqrt{\sum(A_i^2)} \times \sqrt{\sum(B_i^2)}}$$

INTUITION:

Measures the angle between two vectors (ignores magnitude)

$\cos(\theta)$

/

θ (small angle = similar)

/

$\sqrt{\sum(A_i^2)}$

SCALE:

• 1.0 = Identical meaning

• 0.0 = Unrelated

• -1.0 = Opposite meaning

EXAMPLES:

Query: "What is a Spring?"

"What is a Spring in Wyckoff?" Similarity: 0.95

"Explain Spring trading event" Similarity: 0.88

"Phase C accumulation test" Similarity: 0.72

"What is the stock price?" Similarity: 0.25

"Recipe for chocolate cake" Similarity: 0.05

Complete Data Flow

COMPLETE CHATBOT DATA FLOW

USER INPUT

"When should I enter after seeing a Selling Climax?"

STEP 1: EMBED QUERY

"When should I enter after seeing a Selling Climax?"

MiniLM

Query Vector: [0.23, -0.45, 0.12, ..., 0.67] (384 dims)

STEP 2: SEARCH VECTOR DATABASE

Query Vector ChromaDB
(200+ Q&A embeddings)

Calculate cosine similarity with ALL stored embeddings

Sort by similarity, return Top-3:

1. [0.89] Q: "What happens after Selling Climax?"
A: "After SC, look for Automatic Rally..."

2. [0.82] Q: "When to enter in accumulation?"
A: "Best entries are at Spring or LPS..."

3. [0.78] Q: "Entry timing in Phase C?"
A: "Phase C provides test confirmation..."

STEP 3: BUILD AUGMENTED PROMPT

Instruction:

You are an expert on Richard Wyckoff's trading methodology.

Use the provided context to give a detailed, accurate answer.

--- Relevant Wyckoff Knowledge ---

[1] Q: What happens after Selling Climax?
A: After SC, look for Automatic Rally which sets the upper boundary of the trading range. Then wait for Secondary Test to confirm selling is exhausted...

[2] Q: When to enter in accumulation?
A: Best entries are at Spring or Last Point of Support. These provide low-risk entry with clear stop levels...

[3] Q: Entry timing in Phase C?
A: Phase C provides test confirmation. Wait for Spring or successful test of lows before entering...

--- End of Context ---

Question:
When should I enter after seeing a Selling Climax?

Answer:

After observing a Selling Climax (SC), you should NOT enter immediately. The SC marks the stopping of the downtrend, but confirmation is needed. Here's the proper sequence:

1. After SC, wait for the Automatic Rally (AR) - this bounce sets the upper boundary of your trading range.

STEP 4: GENERATE RESPONSE

Augmented Prompt Fine-tuned LLaMA-2-7B
(Base Model + LoRA Adapters)

Generation Settings:
temp=0.7
top_p=0.9
max_tokens=300

GENERATED RESPONSE

"After observing a Selling Climax (SC), you should NOT enter immediately. The SC marks the stopping of the downtrend, but confirmation is needed. Here's the proper sequence:

1. After SC, wait for the Automatic Rally (AR) - this bounce sets the upper boundary of your trading range.

2. Then look for the Secondary Test (ST) - price returns to the SC area with LESS volume, confirming selling is exhausted.

3. The BEST entry comes in Phase C - look for a Spring (false breakdown below support) or Last Point of Support (LPS).

These events provide low-risk entries with clear stop-loss levels just below the Spring or LPS low."

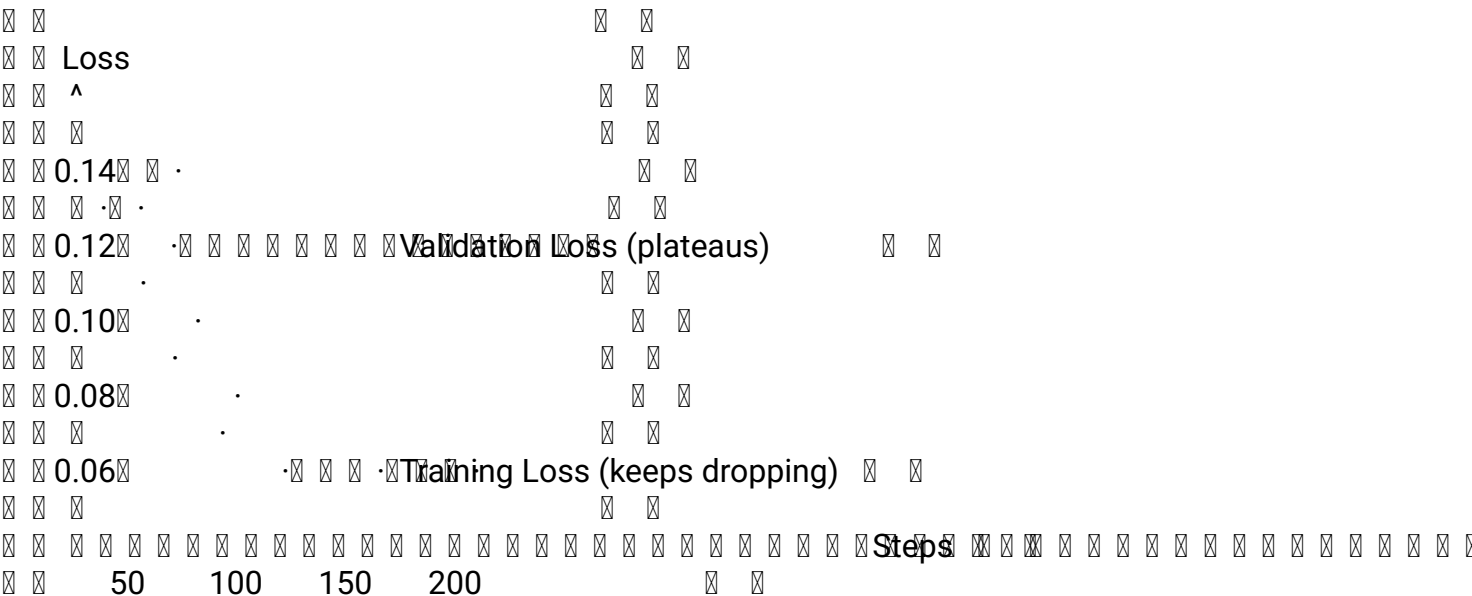
Training Results Analysis

OUR TRAINING RESULTS

METRICS:

Metric	Start	End	Interpretation
Training Loss	0.143	0.062	Good learning
Validation Loss	0.133	0.124	Slight overfit
Training Time	-	15 min	Fast
GPU (A100)	-	~8GB	Efficient

LOSS CURVES:



Loss ^

0.143

0.124

0.100

0.080

0.060

50 100 150 200 Steps

Training Loss (keeps dropping)

Validation Loss (plateaus)

- INTERPRETATION:
- Model learned Wyckoff knowledge well (train loss dropped 56%)
 - Slight overfitting (validation loss plateaued)
 - Could improve with: more data, more dropout, early stopping
 - Overall: GOOD for production use



What User Sees in Streamlit

CHATBOT UI

Wyckoff Trader

AI-Powered Trading Analysis

Chat Window

What is a Spring in Wyckoff?

[User]

Wyckoff Bot

A Spring is a critical Phase C event in Wyckoff's

accumulation schematic. It occurs when price temporarily

breaks below the established support level, only to

quickly recover back into the trading range.

Key characteristics of a successful Spring:

- LOW VOLUME on the breakdown (lack of real selling)

- Quick recovery back above support

- Traps short sellers who sold the "breakdown"

This is one of the BEST entry points because it offers

low risk (stop just below Spring low) and high reward

(potential for entire markup move).

Quick Questions

[What is a Spring?] [Wyckoff's three laws?] [When to enter SC?]

[Accumulation vs Distribution?] [What is Phase C?]

Retrieved Context

Sources used to generate this answer:

[95% match] Q: What is a Spring event?

A: A Spring is a false breakdown below support that...

[88% match] Q: How do I trade a Spring?

A: Enter after price recovers above support with tight stop...

[82% match] Q: What happens in Phase C?

A: Phase C is the test phase where Spring or UTAD occurs...

Ask a question... [Send]

✓ Part I Requirements Checklist

Requirement	What We Built	Status
Fine-tune LLM on domain data	LLaMA-2-7B + LoRA on 200+ Q&A pairs	✓
Build RAG pipeline	MiniLM embeddings + ChromaDB	✓
Combine for better answers	Context-augmented generation	✓
Chat interface	Streamlit with message history	✓
Show retrieved context	Expandable context panel	✓
Quick question buttons	Pre-defined common questions	✓

Key Terminology Quick Reference

Term	Definition
Fine-tuning	Training a pre-trained model on domain-specific data
LoRA	Low-Rank Adaptation - efficient fine-tuning with small adapters

Quantization	Reducing model precision (4-bit) to save memory
RAG	Retrieval Augmented Generation - retrieve then generate
Embedding	Converting text to numerical vector representing meaning
Vector Database	Database optimized for similarity search on vectors
Cosine Similarity	Measure of how similar two vectors are (0 to 1)
Top-K Retrieval	Getting the K most similar items from database
Prompt Engineering	Crafting the input format for best LLM responses
Tokenization	Converting text to numbers the model understands

Code Structure Summary

chatbot.py

```

❑
❑ ❑ CONFIG                # All configuration in one place
❑
❑ ❑ VectorStore           # RAG retrieval component
❑ ❑ ❑ initialize()        # Load embedder + build ChromaDB
❑ ❑ ❑ search()            # Find similar Q&As
❑
❑ ❑ LLMGenerator          # Response generation component
❑ ❑ ❑ initialize()        # Load fine-tuned LLaMA
❑ ❑ ❑ generate()          # Generate text from prompt
❑
❑ ❑ WyckoffChatbot        # Combined system
❑ ❑ initialize()          # Setup both components
❑ ❑ ask()                  # Full RAG + LLM pipeline
❑ ❑ ask_simple()          # RAG only (fast, no GPU)
❑ ❑ get_suggested_questions()

```

*Document Version: 1.0**For GenAI Course - Wyckoff Team*